Technical Specification
4-Dimensional Chess Engine on the Hypercubic Board $[4]^4$

"The art of doing mathematics consists in finding that special case which contains all the germs of generality."
– David Hilbert

Roll No.	**Name**
1012411172	Ananya Bhonsle
1012411180	Nishant Avinash Patil
1012411193	Avadhut Kale
1012411210	Kushal Jadhav
1012411237	Ayush Padwal

April 2026

Contents

1 Mathematical Foundations . . . 3
1.1 The Board as a Finite Hypercube . . . 3
1.1.1 Flat Indexing . . . 3
1.1.2 Board Symmetry . . . 3
1.2 Piece Taxonomy and Move Vectors . . . 3
1.2.1 Axis-Aligned Motion (Rook) . . . 3
1.2.2 Face-Diagonal Motion (Bishop) . . . 3
1.2.3 Combined and Short-Range Pieces . . . 3
1.2.4 The Cat Piece . . . 3
1.3 Game State . . . 3
1.3.1 Zobrist Hashing . . . 4
1.4 Evaluation Function . . . 4
2 Engine Architecture (Nishant Avinash Patil) . . . 5
2.1 Module Diagram . . . 5
2.2 Input / Output Algorithm . . . 5
3 Godot Integration and Movement Logic (Ananya Bhonsle) . . . 6
3.1 Module Diagram . . . 6
3.2 Input / Output Algorithm . . . 6
4 Custom Pieces and Visual Effects (Kushal Jadhav & Ayush Padwal) . . . 7
4.1 Module Diagram . . . 7
4.2 Input / Output Algorithm . . . 7
5 Formal Verification (Avadhut Kale) . . . 8
5.1 Lean 4 Deliverables . . . 8
5.2 Module Diagram . . . 8
5.3 Input / Output Algorithm . . . 9
6 Testing Plan . . . 10
6.1 Unit Tests (GoogleTest) . . . 10
6.2 Integration Tests . . . 10
6.3 Lean Verification . . . 10
7 Appendix: Notation Summary . . . 11

1 Mathematical Foundations

1.1 The Board as a Finite Hypercube

The game is played on a four-dimensional hypercubic lattice, defined as the set of coordinate 4-tuples: $\mathbb{B} = \{(x, y, z, w) \mid x, y, z, w \in \{0, 1, 2, 3\}\}$ The board contains exactly $4^4 = 256$ cells.

1.1.1 Flat Indexing

To optimise memory layout and performance for the C++ Engine, the 4D coordinate (x, y, z, w) is mapped bijectively to a 1D scalar index using the function $\varphi : \mathbb{B} \to \{0, ..., 255\}$: $\varphi(x, y, z, w) = x + 4y + 16z + 64w$

1.1.2 Board Symmetry

The board exhibits deep symmetries corresponding to the hyperoctahedral group B_4. This allows the engine to evaluate symmetric positions identically, vastly reducing the search space via the transposition table.

1.2 Piece Taxonomy and Move Vectors

Pieces in 4D space move along specific vector types defined in $\mathbb{Z}^4$.

1.2.1 Axis-Aligned Motion (Rook)

Rooks move along the 8 axis-aligned unit directions: $(+-1, 0, 0, 0)$, $(0, +-1, 0, 0)$, etc.

1.2.2 Face-Diagonal Motion (Bishop)

Bishops traverse the 24 face-diagonal directions, characterized by changing exactly two coordinates simultaneously by $+-1$.

1.2.3 Combined and Short-Range Pieces

Queens combine Rook and Bishop vectors. Knights leap in an L-shape in 4D, yielding 48 distinct move vectors. Kings step exactly one cell in any valid axis-aligned or diagonal direction.

1.2.4 The Cat Piece

The Cat is a volatile piece that spawns dynamically. It has the unique ability to clear boundary cells, altering the spatial dynamics of the game.

1.3 Game State

The state encompasses piece positions, active turn, castling rights (adapted for 4D), and en-passant targets.

1.3.1 Zobrist Hashing

State caching is managed via Zobrist hashing, mapping the board to a 64-bit integer. A precomputed table $T[p][i]$ stores a string for piece type p at cell i. $Z(S) = \bigoplus_{c \in \mathbb{B}} T[p_c][\varphi(c)]$

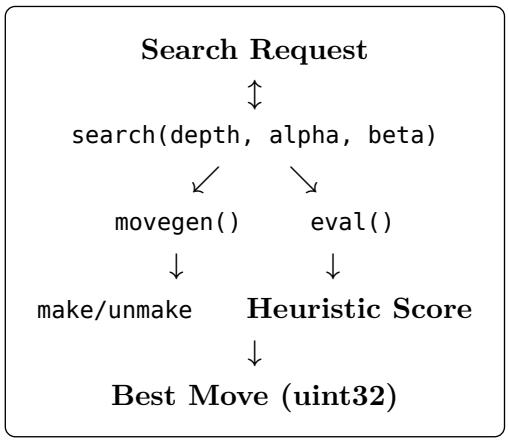
1.4 Evaluation Function

The heuristic evaluation function assigns a static score to a board state, calculating material balance, piece mobility, hyper-centre control, and king safety across the 4D space.

2 Engine Architecture (Nishant Avinash Patil)

The core chess engine is responsible for maintaining state, generating moves, and computing optimal play using minimax algorithms.

2.1 Module Diagram



2.2 Input / Output Algorithm

Input: Board state S, Depth d, α, β
Output: Optimal move score v

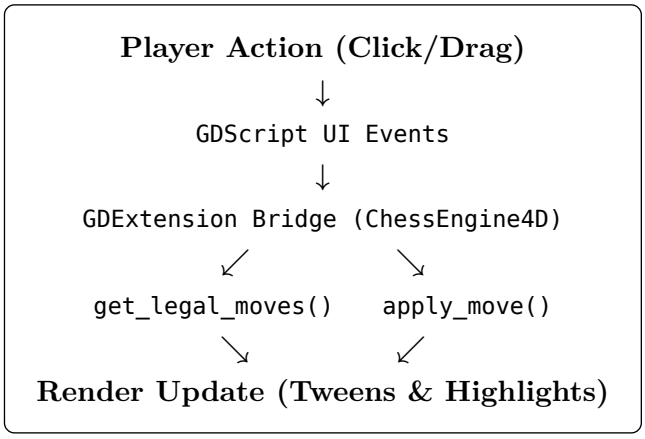
```
if d == 0 or game_over(S) then
   return evaluate(S)
moves ← generate_legal_moves(S)
for m in moves do
   make_move(S, m)
   score ← −search(S, d − 1, −β, −α)
   unmake_move(S, m)
   if score ≥ β then return β
   if score > α then α ← score
return α
```

Algorithm 1: Alpha-Beta Minimax Search Routine

3 Godot Integration and Movement Logic (Ananya Bhonsle)

This module handles the bridge between the C++ engine and the Godot 4 frontend, processing player inputs and returning visual states.

3.1 Module Diagram



3.2 Input / Output Algorithm

Input: Godot Dictionary (Current State), Packed Move
Output: Godot Dictionary (New State), UI Signals

```
function process_player_move(current_state, player_move):
   if not is_legal(player_move): return Error
   new_state ← engine.apply_move(current_state, player_move)
           emit_signal("piece_moved",        player_move.from, player_move.to)
   ai_move ← engine.get_best_move(new_state, depth=4)
   final_state ← engine.apply_move(new_state, ai_move)
   emit_signal("piece_moved", ai_move.from, ai_move.to)
   return final_state
```

Algorithm 2: Godot Bridge State Update

4 Custom Pieces and Visual Effects (Kushal Jadhav & Ayush Padwal)

This module handles the dynamic generation of special entities (Cat and Phantom) and renders their corresponding visual effects in 3D space.

4.1 Module Diagram

Turn Start Trigger

↓

`RNG Check (cat_event / phantom_event)`

↓

`Board Mutation (Entity Placed/Removed)`

↓

`Emit Signals (cat_spawned / phantom_spawned)`

↓

VFX Trigger (GPUParticles3D / Screen Shaders)

4.2 Input / Output Algorithm

Input: Board State S, Turn Number t, RNG Seed
Output: Mutated State S', Emitted Godot Signals

```
function trigger_cat_event(S):
   if random() > CAT_PROBABILITY: return false
   empty_cells ← get_empty(S)
   cell ← select_random(empty_cells)
   place_piece(S, CAT, cell)
   boundary_enemies ← get_enemies_on_boundary(S)
   for enemy in boundary_enemies:
      remove_piece(S, enemy)
   emit_signal("trigger_vfx", "CAT_BURST", cell)
   return true
```

Algorithm 3: Dynamic Entity Spawning & VFX

5 Formal Verification (Avadhut Kale)

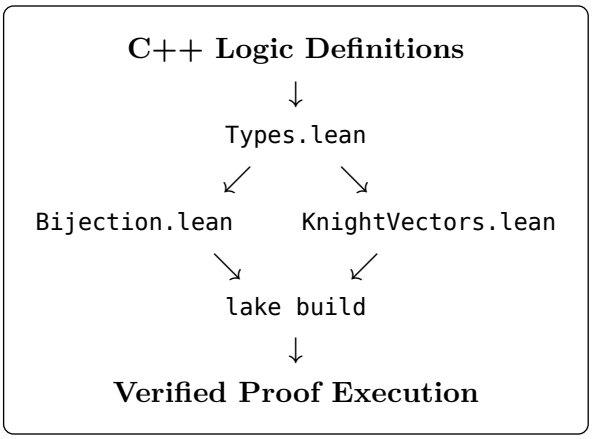
The formal verification phase ensures the mathematical correctness of the engine's core indexing and movement generation logic using the Lean 4 theorem prover.

5.1 Lean 4 Deliverables

The following files and proofs must be implemented:

- `Types.lean`: Formalise the definitions of the board, the flat-index bijection, piece types, and move types using Lean 4 structures.
- `Bijection.lean`: Prove the exact inverse properties of the flat-index mapping:
  - Prove $\varphi^{-1}(\varphi(x)) = x$ for all cells.
  - Prove $\varphi(\varphi^{-1}(i)) = i$ for all valid flat indices.
- `KnightVectors.lean`: Prove that the total count of valid knight vectors is exactly 48 via decidable computation over an explicit enumeration.
- `BoundaryCount.lean`: Prove that the total number of boundary cells is 240, relying on the inclusion-exclusion mathematical argument.
- `MovegenTermination.lean`: Prove the termination (well-foundedness) of the ray-traversal recursion function, bounding it by the board's finite dimension.

5.2 Module Diagram



5.3 Input / Output Algorithm

Input: C++ Engine Logic Specifications
Output: Mechanised Lean 4 Proof Verification

```
1. Define mathematical Types and Bounds in Lean.
2. Map C++ flat-index calculations to Lean functions.
3. State theorem hypotheses (e.g., bijection identity).
4. Construct proofs using Lean core tactics (simp, rw, intro).
5. Execute lake build against the directory.
6. IF build == Success THEN proofs are valid.
7. ELSE return compilation trace and fix tactics.
```

Algorithm 4: Formal Verification Pipeline

6 Testing Plan

6.1 Unit Tests (GoogleTest)

- **Engine (Nishant)**: $\varphi\varphi^{-1}$ round-trip for all 256 indices; `make`/`unmake` identity on 1000 random positions; Zobrist incremental vs full agreement; legal move count from initial position at depth 1; `perft(d) = sum_m perft(d-1)` consistency check.
- **Custom pieces (Kushal/Ayush)**: `evaluate` returns 0 on symmetric positions; Cat does not spawn on occupied cells; swat removes exactly the boundary enemy pieces; Phantom never placed on $\partial\mathbb{B}$; search finds mate-in-1 on hand-crafted positions.

6.2 Integration Tests

Full game loop: initial state → legal moves → apply move → engine best move → repeat. Run 100 random self-play games without crashes or illegal states. Cat and Phantom events must fire at statistically plausible rates (verified by chi-squared test on 10,000 turns).

6.3 Lean Verification

`lake build` in `lean/` must pass on the CI pipeline. This is a hard acceptance criterion for project completion.

7 Appendix: Notation Summary

Symbol	Meaning
$\mathbb{B}$	The board $\{0,1,2,3\}^4$, 256 cells
φ	Flat-index bijection $\mathbb{B} \to \{0,...,255\}$
$\partial\mathbb{B}$	Boundary cells; $\lvert\partial\mathbb{B}\rvert = 240$
$\mathcal{A}$	8 axis-aligned unit directions in $\mathbb{Z}^4$
$\mathcal{F}$	24 face-diagonal unit directions in $\mathbb{Z}^4$
$Z(S)$	64-bit Zobrist hash of state S
$\mathrm{MM}(n,d)$	Minimax value of node n at depth d
(α,β)	Alpha-beta pruning bounds